

1.3.1 Function Definition

A function is declared in the following manner:

return-type function-name (parameter-list,...) { body... }

return-type is the variable type that the function returns. This can not be an array type or a function type. If not given, then `int` is assumed.

function-name is the name of the function.

parameter-list is the list of parameters that the function takes separated by commas. If no parameters are given, then the function does not take any and should be defined with an empty set of parenthesis or with the keyword `void`. If no variable type is in front of a variable in the parameter list, then `int` is assumed. Arrays and functions are not passed to functions, but are automatically converted to pointers. If the list is terminated with an ellipsis (`, . . .`), then there is no set number of parameters. Note: the header `stdarg.h` can be used to access arguments when using an ellipsis.

If the function is accessed before it is defined, then it must be prototyped so the compiler knows about the function. Prototyping normally occurs at the beginning of the source code, and is done in the following manner:

return-type function-name (parameter-type-list) ;

return-type and *function-name* must correspond exactly to the actual function definition. *parameter-type-list* is a list separated by commas of the types of variable parameters. The actual names of the parameters do not have to be given here, although they may for the sake of clarity.

Examples:

```
int joe(float, double, int);
```

This defines the prototype for function joe.

```
int joe(float coin, double total, int sum)
{
    /*...*/
}
```

This is the actual function joe.

```
int mary(void), *lloyd(double);
```

This defines the prototype for the function mary with no parameters and return type `int`. Function lloyd is defined with a double type parameter and returns a pointer to an `int`.

```
int (*peter)();
```

Defines peter as a pointer to a function with no parameters specified. The value of peter can be changed to represent different functions.

```
int (*aaron(char *(*)(void)) (long, int);
```

Defines the function aaron which returns a pointer to a function. The function aaron takes

one argument: a pointer to a function which returns a character pointer and takes no arguments. The returned function returns a type `int` and has two parameters of type `long` and `int`.

1.3.2 Program Startup

A program begins by calling the function `main`. There is no prototype required for this. It can be defined with no parameters such as:

```
int main(void) { body... }
```

Or with the following two parameters:

```
int main(int argc, char *argv[]) { body... }
```

Note that they do not have to be called `argc` or `argv`, but this is the common naming system.

`argc` is a nonnegative integer. If `argc` is greater than zero, then the string pointed to by `argv[0]` is the name of the program. If `argc` is greater than one, then the strings pointed to by `argv[1]` through `argv[argc-1]` are the parameters passed to the program by the system.

Example:

```
#include<stdio.h>

int main(int argc, char *argv[])
{
    int loop;

    if(argc>0)
        printf("My program name is %s.\n",argv[0]);

    if(argc>1)
    {
        for(loop=1;loop<argc;loop++)
            printf("Parameter #%i is %s.\n",loop,argv[loop]);
    }
}
```